

Analyse des principes de conception (TP)

1 Analyse par principe

1.1 Principe de responsabilité unique (SRP)

Non respecté.

La classe `GestionnaireEmploiDuTemps` gère à la fois la gestion des cours (ajout, modification) et la notification des observateurs : elle a plusieurs responsabilités.

1.2 Principe ouvert / fermé (OCP)

Respecté (partiellement).

Le pattern `Decorator` permet d'ajouter des fonctionnalités aux cours sans modifier la classe de base. En revanche, certaines modifications (p.ex. nouvelles stratégies de notification) peuvent encore demander des changements dans le gestionnaire.

1.3 Principe de substitution de Liskov (LSP)

Respecté.

Les décorateurs et les implémentations concrètes implémentent `ICours` et peuvent être substitués sans casser les comportements attendus (à condition que les décorateurs respectent bien toutes les méthodes de l'interface).

1.4 Principe de ségrégation des interfaces (ISP)

Respecté.

Les interfaces sont minimales et ciblées (`ICours`, `Observer`, `Subject`) : les classes ne sont pas forcées d'implémenter des méthodes inutiles.

1.5 Principe d'inversion des dépendances (DIP)

Non respecté.

Le `CoursBuilder` dépend d'une implémentation concrète (`Cours`) plutôt que d'une abstraction (`ICours`), ce qui crée un couplage fort et réduit la flexibilité.

1.6 Composition over Inheritance (COI)

Partiellement non respecté.

Le Decorator utilise la composition mais la hiérarchie des décorateurs repose aussi sur l'héritage de `CoursDecorator` ; certains comportements auraient gagné à être entièrement basés sur la composition pour éviter des dépendances d'héritage inutiles.

1.7 Loi de Déméter

Respectée.

Les classes communiquent principalement avec leurs dépendances directes et il n'existe pas d'enchaînements d'appels longs dans le code fourni.